

Handoff #8: re-run the cache guard

Since #7 the cache guard changed in two ways: deciders now compare the two queries only, and the vCache labels were corrected. Every cache-guard decider has to be run again. One command does it, on Gilbreth.

TL;DR

- What you do: `git pull`, then `bash scripts/cacheguard_v2_submit.sh`. It queues 11 jobs (API deciders, Kev-4B, Qwen3-8B, floor retrain) and one analysis job that waits for them. Then commit and push the results.
- Cost: OpenRouter only (Jev, GPT-OSS-20B, GPT-5.6). The last full run was \$23.7 with longer prompts; this one should be less. Hard caps: about \$18 per dataset, \$54 total.
- Change 1, the question: deciders no longer see the cached answer. They judge whether the new query and the cached query are the same request. That is now a stated assumption of the paper.
- Change 2, the labels: Claude Opus 5.5 re-judged all 5,200 LMArena + SearchQueries pairs blind, in two passes. 156 LMArena (6.0%) and 409 SearchQueries (15.7%) labels changed. GSM-Plus untouched.
- Why it matters: on the old runs, correcting the labels alone lifts every decider on SearchQueries from ~71% to 82-84%. The 71% ceiling was the labels, not the models.

Your first 5 minutes

```
git checkout main && git pull # needs commit 6ffa76c or later
source scripts/env.sh && python3 scripts/check_connectivity.py # OpenRouter reachable, key set
bash scripts/cacheguard_v2_submit.sh # queues everything, prints job ids
squeue -u $USER # watch; logs in results/logs/
```

What the script submits

job	per dataset (lmarena, searchqueries, gsmplus)	where	\$
cacheguard_api.slurm	Jev x5 (test) + x5 (dev), GPT-OSS-20B x5, GPT-5.6 x3, then analyze_cacheguard.py	A100 node, API calls	capped: Jev \$1+\$1, OSS \$2, GPT-5.6 \$14
kev_replay.slurm	Kev-4B x5 on test, energy measured (KEV_MODEL=kev-4b)	A100, local	0
llm_local_replay.slurm	Qwen3-8B x5 on test via vLLM, energy measured	A100, local	0
cacheguard_floor.slurm (once)	floor classifier retrained on the corrected dev labels, all 3 datasets	A100	0
dwg-cacheguard-v2-analysis (once)	waits for all the above; per-dataset tables, compare_cacheguard.py (paired CIs), figures 15-16	A100	0

- Fresh start: the old decider runs moved to results/replay/cacheguard-<dataset>/v1-answer-question/. The analysis reads only the parent folder, so they are not counted.
- Resumable: if a job dies (time limit, 402, network), run the same script again: finished calls are kept and only the missing ones run. To redo one dataset: DATASETS=lmarena bash scripts/cacheguard_v2_submit.sh.
- Kept as they were: cross-encoder scores, vCache scores and the measured GPT-5.6 regeneration cost. They do not depend on the question. The floor is retrained because its dev labels changed.
- Local jobs get 4 h each (LOCAL_TIME=06:00:00 bash ... to raise it). Last round Qwen3-8B took ~0.4 s and Kev ~0.1 s per decision, 10,000 decisions per dataset.

When everything has finished

```
less results/replay/cacheguard-lmarena/report/summary.md # also searchqueries, gsmplus
less results/replay/cacheguard-paired.md # paired differences with CIs
grep -h "failures" results/logs/dwg-cacheguard-*.out | tail # failure counts should be ~0
git add results/replay/cacheguard-* results/replay/cacheguard-paired.md
git commit -m "Cache guard v2: query-only question on corrected labels" && git push
```

Check before committing: every decider has its full count in each summary (Jev, GPT-OSS, Kev-4B, Qwen3-8B: 10,000 decisions; GPT-5.6: 6,000), and the failures column is near zero.

Change 1 · the cache guard decides on the queries only

Deciders now see two things: the new query and the cached query. They answer whether these are the same request. If they are, the cache returns the stored answer as it is; nobody checks that answer. The paper states this as an assumption, for three reasons:

- Cost: reading the stored answer costs most of what the cache saves (LMArena answers run to thousands of tokens).
- Correctness belongs elsewhere: to the model that wrote the answer, and to the cache's TTL. With a TTL fitted to the use case the answer is likely still right; with the wrong one even a perfect answer goes stale ("today's highest temperature in London" cached for a week is wrong the next day, whatever model wrote it).
- Disagreement is not error: a decider that finds a stored answer incomplete or badly formatted, even one written by the same or another frontier model, is two experts disagreeing, not a wrong cache hit.

How we found it: under the old question ("reuse only if it fully and correctly answers"), GPT-5.6 said regenerate on 24% of LMArena pairs whose two prompts were identical. It was grading the stored answers, e.g. "Abigail has 7 sisters and 9 brothers; how many sisters does each brother have?" stored as 7 (correct: 8).

Change 2 · corrected vCache labels

dataset	pairs reviewed	flip candidates	changed	to reuse	to regenerate
LMArena	2,600	174	156 (6.0%)	122	34
SearchQueries	2,600	465	409 (15.7%)	267	142

- How: a guideline written first; pass 1 judged every pair blind (no label, no similarity, no decider output); a separate blind pass 2 re-judged each high-confidence disagreement; a label changed only when both agreed.
- Who: Claude Opus 5.5, an LLM. No human changed a label. A human spot-check is still to do (below).
- Why vCache's labels fail: SearchQueries groups are a transitive closure of GPT-4.1-nano pair judgements ($A \sim B$ and $B \sim C$ make $A \sim C$, though A's answer was written for A; small differences add up; one wrong link merges whole groups). LMArena variants inherit their seed's label with no check that they still ask the same thing.
- Examples: "count blank cells" vs "count non blank cells" was same; "best workout plans for women" vs "best workout plan for women" was different; CompTIA SY0-501 vs SY0-701 was same.
- Where it lives: streams keep `reuse_correct_original` and `label_source`; every change with both passes' reasons is in `results/label-corrections/`, whose README is the paper write-up.

Preview: the old runs re-scored on the corrected labels (test accuracy)

decider	SearchQueries: vCache -> corrected	LMArena: vCache -> corrected
Jev	0.713 -> 0.836	0.712 -> 0.747
GPT-5.6	0.710 -> 0.829	0.706 -> 0.724
GPT-OSS-20B	0.717 -> 0.822	0.773 -> 0.814
Kev-4B	0.707 -> 0.844	0.869 -> 0.911
Qwen3-8B	0.721 -> 0.835	0.856 -> 0.903

LMarena barely moves for Jev and GPT-5.6 because the old runs still graded the stored answers. The re-run with the query-only question is what tests that.

Gotchas

- vCache is scored on its own labels. vCache can serve a different cached entry than the one we relabelled, so its correctness still comes from vCache's classes. Say so next to its numbers.
- Do not re-run the label correction. The streams in git are already corrected; `apply_cacheguard_label_corrections.py` is idempotent, but there is no need to run it.
- GSM-Plus labels did not change (our exact numeric answers), but its deciders are re-run because the question changed.
- OpenRouter balance: each job stops paid calls at its cap or at the first 402; top up and re-run the script to resume.
- Kev and Qwen run one call at a time on a GPU of their own, for the energy numbers. Do not raise their workers.
- Two tests in `tests/test_labels.py` fail on machines without the `tau2` data files. That is unrelated to this change.

What's next (after your run)

- Human spot-check of the corrected labels: a person labels a random sample (e.g. 50 changed + 50 unchanged pairs per dataset) without seeing either label; report agreement with the corrected and with the original labels.
- Update the paper numbers (cache-guard tables, figures 15–16, paired claims) from the new reports.
- Paper text: the assumption above, and the label write-up in `results/label-corrections/README.md` (source and citations, how we found it, why vCache's method fails, how ours fixes it, who corrected).

New and changed files

```
src/dwg/cacheguard.py # query-only question
results/label-corrections/ # GUIDELINE.md, README.md, pass1/pass2/corrections per dataset
scripts/apply_cacheguard_label_corrections.py # applies the two-pass rule
scripts/cacheguard_v2_submit.sh # this run
results/replay/cacheguard-*/v1-answer-question/ # archived old runs
DECISIONS.md # rows dated 2026-10-07
```